

Module 3

Testbench Fundamentals (Verilog and C++)

Learning objectives

- Understand testbench architecture and basic verification concepts for both Verilog and C++ testbench...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Verification Fundamentals"]

T2["Testbench Architecture"]

T1 --> T2

T3["Basic Testbench Components"]

T2 --> T3

Understand testbench architecture and basic verification concepts for both Verilog and C++ testbenches

Overview

- This module introduces the fundamental concepts of testbench design for both Verilog (iverilog) and...

Design architecture

1. Canonical testbench-DUT boundary

- DUT: Device Under Test in
`module3/dut/` (gates,
multiplexers, counters)
- TB wrapper: Encapsulates
instantiation, clock/reset, and
test program only
- Signals: Explicit wires/regs
(Verilog) or model ports (C++) —
no hidden coupling
- Lifecycle: Build → reset → run
tests → report → `\$finish` /
`return` exit code

```
Rtl Architecture  
  
Diagram: Rtl Architecture  
(render with mmdc when available)  
  
flowchart TB  
  subgraph paradigms["Dual testbench paradigms"]  
    VTB[Verilog TB → iverilog → vvp]  
    CTB[C++ TB → verilator → obj_dir]  
  end  
  subgraph dut3["module3/dut — same RTL"]
```

2. Dual-paradigm environment architecture

- Verilog TB (``examples/verilog_testbenches/``): Event-driven
``initial`/`always``
- C++ TB
(``examples/cpp_testbenches/``):
``eval()`` loop with Verilator model
- Shared DUT: Same RTL verified two ways — highlights tool-specific TB idioms
- Comparison guide:
``examples/comparison/`` documents mapping between styles

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
    subgraph vpath["Verilog path"]
        VSTIM[initial blocks] --> VDUT[DUT]
        VDUT --> VCHK[$display checks]
    end
    subgraph cpath["C++ path"]
```


3. Verification component roles (UVM-aligned)

- Driver: Applies stimulus
(assignments before `eval` or
`#delay`)
- Monitor: Samples outputs; logs
transactions or pin values
- Scoreboard: Compares actual vs
expected; increments pass/fail
- Clock/reset agents: Reusable
blocks generating periodic clock
and reset sequences

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
  VRUN[verilog_testbenches make] --> CRUN[cpp_testbenches make]
  CRUN --> DIFF[comparison side-by-side]
  DIFF --> PICK[choose tool by need]
  PICK --> REG[module3.sh --check]
```

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph paradigms["Dual testbench paradigms"]

VTB[Verilog TB → iverilog → vvp]

CTB[C++ TB → verilator → obj_dir]

end

subgraph dut3["module3/dut — same RTL"]

Module 3: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

subgraph vpath["Verilog path"]

VSTIM[initial blocks] --> VDUT[DUT]

VDUT --> VCHK[\$display checks]

end

subgraph cpath["C++ path"]

Module 3: stimulus, observation, and checking.

Verilog TB Makefile — counter_test

```
counter_test: counter_test.v
    @echo "=== Compiling counter_test ==="
    $(IVERILOG) -o counter_test counter_test.v
$(DUT_DIR_MODULE1)/counters/counter_4bit.v
    @echo "=== Running simulation ==="
    $(VVP) counter_test
```

Verilog TB — clock and reset

```
module counter_test;

    //
    =====
    =
    // TESTBENCH SIGNAL DECLARATIONS
    //
    =====
    =
    // Clock signal - driven by always block (continuous clock
generation)
    reg          clk;

    // Reset signal (active-low: 0 = reset, 1 = normal operation)
    // Active-low resets are common in digital design
    reg          rst_n;

    // Enable signal (1 = count, 0 = hold current value)
```

Shared DUT — counter_4bit

```
/**
 * 4-bit Counter Design Under Test (DUT)
 *
 * This is a synchronous 4-bit counter with reset and enable, used for
learning
 * testbench development. This module demonstrates sequential logic,
which requires
 * clock-based testbench patterns and is more complex to verify than
combinational logic.
 *
 * Verification Context:
 * - Sequential logic requires clock generation and timing control in
testbenches
 * - Demonstrates reset sequences, enable control, and state
observation
 * - In UVM, sequential logic like counters are used to learn
transaction-based
 * verification, clocking blocks, and sequence generation
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- `sim_main.cpp`: generates `clk/rst_n`, calls `eval()` until `$finish`
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: `module3/examples/<example>/` — run `make run` from that directory

Directed test execution sequence (1)

- Both paradigms count errors locally
- same PASS criteria without manual waveform grading
- Instantiate DUT in Verilog TB
- iverilog compile
- vvp

Directed test execution sequence (2)

- repeat with C++ TB and Verilator
- review comparison/ mapping guide
- `cd module3/examples/verilog_testbenches && make counter_test`

C++ TB Makefile — Verilator build

```
counter_test: counter_test.cpp
    @echo "=== Compiling counter_test ==="
    $(VERILATOR) $(VERILATOR_FLAGS) -I$(DUT_DIR_MODULE1)/counters \
        $(DUT_DIR_MODULE1)/counters/counter_4bit.v
counter_test.cpp
    @echo "=== Running simulation ==="
    ./obj_dir/Vcounter_4bit
```

C++ TB — DUT init and reset test

```
int main(int argc, char** argv) {  
    //
```

```
=====
```

```
=
```

```
    // VERILATOR INITIALIZATION  
    //
```

```
=====
```

```
=
```

```
    Verilated::commandArgs(argc, argv);
```

```
    //
```

```
=====
```

```
=
```

```
    // TEST HEADER  
    //
```

```
=====
```

```
=
```

```
    std::cout << "=====
```

Verification & testing methods

1. Structured test flow (build-connect-run)

- Build: Compile RTL + TB;
elaborate hierarchy
- Connect: Port map DUT; tie off
unused inputs
- Run: Execute directed cases (AND
truth table, counter wrap, MUX
exhaust)
- Cleanup: Close files, print
summary, non-zero exit on
failure

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
VRUN[verilog_testbenches make] --> CRUN[cpp_testbenches make]
CRUN --> DIFF[comparison side-by-side]
DIFF --> PICK[choose tool by need]
PICK --> REG[module3.sh --check]

2. Directed and exhaustive patterns

- Truth-table tests: All input combinations for small combinational DUTs
- Sequential tests: Known count sequences after reset deassert
- Assertions: `assert` (C++) or immediate checks (Verilog) on each vector

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
VRUN[verilog_testbenches make] --> CRUN[cpp_testbenches make]
CRUN --> DIFF[comparison side-by-side]
DIFF --> PICK[choose tool by need]
PICK --> REG[module3.sh --check]

3. Cross-paradigm verification discipline

- Same spec, two TBs: Document expected behavior once; implement in Verilog and C++
- Self-checking: Both paradigms count errors locally — no manual waveform grading
- Orchestration:
`./scripts/module3.sh --verilog-testbenches` vs `--cpp-testbenches`

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
VRUN[verilog_testbenches make] --> CRUN[cpp_testbenches make]
CRUN --> DIFF[comparison side-by-side]
DIFF --> PICK[choose tool by need]
PICK --> REG[module3.sh --check]

Paradigm comparison checklist

```
// missing source file: module3/examples/comparison/README.md
```

Syllabus topics

Protocol & design details

1. Verification Fundamentals (1/5)

- What is verification?
- Ensuring design correctness
- Finding bugs before production
- Verification vs. validation
- Testbench Purpose and Role
- Stimulus generation

1. Verification Fundamentals (2/5)

- Response monitoring
- Result checking
- Testbench as verification environment
- Design Under Test (DUT) Concept
- What is DUT?
- DUT instantiation

1. Verification Fundamentals (3/5)

- DUT interface
- DUT testing
- Verification vs. Validation
- Verification: Does it work correctly?
- Validation: Does it meet requirements?
- Differences and overlap

1. Verification Fundamentals (4/5)

- Verification Flow and Methodology
- Test plan creation
- Testbench development
- Test execution
- Result analysis
- Verilog Testbench vs. C++ Testbench Comparison

1. Verification Fundamentals (5/5)

- Similarities
- Differences
- When to use each
- Performance considerations

2. Testbench Architecture (1/6)

- Verilog Testbench Module Structure
- Module declaration
- Signal declarations
- DUT instantiation
- Test sequence
- Module-based organization

2. Testbench Architecture (2/6)

- C++ Testbench Structure
- Main function
- Verilator initialization
- DUT instantiation
- Test sequence
- Class-based organization (optional)

2. Testbench Architecture (3/6)

- DUT Instantiation (Both Paradigms)
- Verilog: Module instantiation
- C++: Object creation
- Signal connectivity
- Port mapping
- Signal Connectivity

2. Testbench Architecture (4/6)

- Input signals
- Output signals
- Bidirectional signals
- Signal naming
- Testbench Hierarchy
- Top-level testbench

2. Testbench Architecture (5/6)

- DUT hierarchy
- Testbench components
- Organization patterns
- Top-Level Testbench Organization
- Single testbench file
- Multiple file organization

2. Testbench Architecture (6/6)

- Include files
- Library organization
- ``module3/examples/verilog_testbenches/`` - Verilog testbench structure
- ``module3/examples/cpp_testbenches/`` - C++ testbench structure

3. Basic Testbench Components (1/6)

- Clock Generation (Verilog and C++)
- Verilog: Always blocks for continuous clock
- C++: Simulation loop for clock toggling
- Clock period control
- Multiple clocks
- Reset Generation (Verilog and C++)

3. Basic Testbench Components (2/6)

- Verilog: Initial blocks for reset sequences
- C++: Reset in C++ code
- Synchronous reset
- Asynchronous reset
- Reset timing
- Stimulus Generation (Both Paradigms)

3. Basic Testbench Components (3/6)

- Test vector generation
- Pattern application
- Sequential stimulus
- Timing control
- Response Monitoring (Both Paradigms)
- Output monitoring

3. Basic Testbench Components (4/6)

- Real-time monitoring
- Post-processing
- Monitoring strategies
- Result Checking (Both Paradigms)
- Expected value calculation
- Output comparison

3. Basic Testbench Components (5/6)

- Error detection
- Pass/fail reporting
- Simulation Control
- Starting simulation
- Stopping simulation
- Simulation termination

3. Basic Testbench Components (6/6)

- Control flow
- ``counter_test.v`` / ``counter_test.cpp`` - Clock and reset generation

4. Signal Access and Monitoring (1/5)

- Verilog: Signal Reading and Writing
- Direct signal access
- Signal assignment
- Signal reading
- Signal types
- C++: Signal Access via Verilator API

4. Signal Access and Monitoring (2/5)

- DUT object access
- Signal reading: ``dut->signal``
- Signal writing: ``dut->signal = value``
- Signal types
- Verilog: `$display`, `$monitor`, `$strobe`
- ``$display``: Immediate output

4. Signal Access and Monitoring (3/5)

- ``$monitor``: Continuous monitoring
- ``$strobe``: End-of-time-step output
- Formatting options
- C++: `std::cout`, Logging Libraries
- ``std::cout`` for output
- Formatting with `iomanip`

4. Signal Access and Monitoring (4/5)

- Logging libraries
- File output
- Formatting Output (Both Paradigms)
- Verilog: Format specifiers
- C++: Stream manipulators
- Table formatting

4. Signal Access and Monitoring (5/5)

- Readable output
- Timing of Display Statements
- When output occurs
- Event scheduling
- Timing considerations
- Debugging output

5. Simulation Control (1/5)

- Verilog: Time Management (# delays)
- Delay control: `#10`
- Time advancement
- Event scheduling
- Timing accuracy
- C++: Time Management (Simulation Loop)

5. Simulation Control (2/5)

- Simulation loop
- Time tracking
- Clock cycle counting
- Time advancement
- Event Scheduling Concepts
- Event-driven simulation

5. Simulation Control (3/5)

- Event queue
- Scheduling order
- Timing relationships
- Simulation Termination
- Verilog: ``$finish``
- C++: ``return`` statement

5. Simulation Control (4/5)

- Cleanup procedures
- Resource management
- Debugging Basics (Both Tools)
- Compilation debugging
- Runtime debugging
- Signal inspection

5. Simulation Control (5/5)

- Waveform analysis
- ``counter_test.v`` / ``counter_test.cpp`` - Time management

6. Simple Verification Patterns (1/5)

- Directed Testing (Both Paradigms)
- Specific test cases
- Known input/output pairs
- Exhaustive testing
- Corner case testing
- Test Vector Application

6. Simple Verification Patterns (2/5)

- Test vector format
- Vector application
- Sequential application
- Batch testing
- Expected vs. Actual Comparison
- Expected value calculation

6. Simple Verification Patterns (3/5)

- Actual value reading
- Comparison logic
- Error detection
- Pass/Fail Reporting
- Test result tracking
- Pass/fail determination

6. Simple Verification Patterns (4/5)

- Result reporting
- Summary generation
- Basic Error Detection
- Error detection methods
- Error reporting
- Error handling

6. Simple Verification Patterns (5/5)

- Debugging aids

7. Paradigm Comparison

- Side-by-Side Comparison Table

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --help
```

Terminal: module3_check

Usage: ./scripts/module3.sh [OPTIONS]

Module 3: Testbench Fundamentals (Verilog and C++)

This script runs examples and tests for Module 3, comparing both paradigms.

OPTIONS:

Examples:

--verilog-testbenches	Run Verilog testbench examples
--cpp-testbenches	Run C++ testbench examples
--comparison	Show comparison guide
--all-examples	Run all examples (default)
--skip-examples	Skip all examples

Tests:

--verilog-tests	Run Verilog testbenches
--cpp-tests	Run C++ testbenches
--all-tests	Run all tests (default)
--skip-tests	Skip all tests

Environment:

--jobs N	Number of parallel build jobs (default: 8)
--no-clean	Skip cleaning before build (faster rebuilds)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all examples and tests (both paradigms)
./scripts/module3.sh
```

```
# Run only Verilog testbenches
./scripts/module3.sh --verilog-testbenches
```

```
# Run only C++ testbenches
./scripts/module3.sh --cpp-testbenches
```

Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Demo: Verilog testbenches

```
$ cd module3/examples/verilog_testbenches && make all
```

Terminal: ex01_verilog_testbenches

```
=== Compiling and_gate_test ===
iverilog -o and_gate_test and_gate_test.v ../../module0/dut/simple_gates/and_gate.v
=== Running simulation ===
vvp and_gate_test
=====
AND Gate Verilog Testbench
=====
Time | a | b | y | Expected | Pass/Fail
-----|---|---|---|-----|-----
VCD info: dumpfile and_gate_test.vcd opened for output.
5000 | 0 | 0 | 0 | 0 | PASS
10000 | 0 | 1 | 0 | 0 | PASS
15000 | 1 | 0 | 0 | 0 | PASS
20000 | 1 | 1 | 1 | 1 | PASS
=====
All tests completed!
=====
and_gate_test.v:158: $finish called at 30000 (1ps)
=== Compiling mux_4to1_test ===
iverilog -o mux_4to1_test mux_4to1_test.v ../../module1/dut/multiplexers/mux_4to1.v
=== Running simulation ===
vvp mux_4to1_test
=====
4-to-1 Multiplexer Verilog Testbench
=====
VCD info: dumpfile mux_4to1_test.vcd opened for output.
Test 1: sel=00, out=1 (expected in0=1) [PASS]
Test 2: sel=01, out=1 (expected in1=1) [PASS]
Test 3: sel=10, out=1 (expected in2=1) [PASS]
Test 4: sel=11, out=1 (expected in3=1) [PASS]
=====
All tests completed!
=====
mux_4to1_test.v:85: $finish called at 30000 (1ps)
=== Compiling counter_test ===
iverilog -o counter_test counter_test.v ../../module1/dut/counters/counter_4bit.v
```


Demo: C++ testbenches

```
$ ls -la module3/examples/cpp_testbenches
```

Terminal: ex02_cpp_testbenches

```
total 32
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu 1172 May 28 12:19 Makefile
-rwxrwxrwx 1 yongfu yongfu 8380 May 28 12:19 and_gate_test.cpp
-rwxrwxrwx 1 yongfu yongfu 9277 May 28 12:19 counter_test.cpp
-rwxrwxrwx 1 yongfu yongfu 3488 May 28 12:19 mux_4to1_test.cpp
```

Demo: Paradigm comparison

```
$ ls -la module3/examples/comparison
```

Terminal: ex03_comparison

```
total 8
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu 4661 May 28 12:19 comparison_guide.md
```

Key concepts

1. Canonical testbench-DUT boundary

- DUT: Device Under Test in ``module3/dut/`` (gates, multiplexers, counters)
- TB wrapper: Encapsulates instantiation, clock/reset, and test program only
- Signals: Explicit wires/regs (Verilog) or model ports (C++) — no hidden coupling
- Lifecycle: Build → reset → run tests → report → ``$finish`` / ``return`` exit code

2. Dual-paradigm environment architecture

- Verilog TB (``examples/verilog_testbenches/``): Event-driven ``initial`/`always``
- C++ TB (``examples/cpp_testbenches/``): ``eval()`` loop with Verilator model
- Shared DUT: Same RTL verified two ways — highlights tool-specific TB idioms
- Comparison guide: ``examples/comparison/`` documents mapping between styles

3. Verification component roles (UVM-aligned)

- Driver: Applies stimulus (assignments before ``eval`` or ``#delay``)
- Monitor: Samples outputs; logs transactions or pin values
- Scoreboard: Compares actual vs expected; increments pass/fail
- Clock/reset agents: Reusable blocks generating periodic clock and reset sequences

1. Structured test flow (build-connect-run)

- Build: Compile RTL + TB; elaborate hierarchy
- Connect: Port map DUT; tie off unused inputs
- Run: Execute directed cases (AND truth table, counter wrap, MUX exhaust)
- Cleanup: Close files, print summary, non-zero exit on failure

2. Directed and exhaustive patterns

- Truth-table tests: All input combinations for small combinational DUTs
- Sequential tests: Known count sequences after reset deassert
- Assertions: ``assert`` (C++) or immediate checks (Verilog) on each vector

3. Cross-paradigm verification discipline

- Same spec, two TBs: Document expected behavior once; implement in Verilog and C++
- Self-checking: Both paradigms count errors locally — no manual waveform grading
- Orchestration: ``./scripts/module3.sh --verilog-testbenches`` vs ``--cpp-testbenches``

1. Verification Fundamentals (1/5)

- What is verification?
- Ensuring design correctness
- Finding bugs before production
- Verification vs. validation
- Testbench Purpose and Role
- Stimulus generation

1. Verification Fundamentals (2/5)

- Response monitoring
- Result checking
- Testbench as verification environment
- Design Under Test (DUT) Concept
- What is DUT?
- DUT instantiation

1. Verification Fundamentals (3/5)

- DUT interface
- DUT testing
- Verification vs. Validation
- Verification: Does it work correctly?
- Validation: Does it meet requirements?
- Differences and overlap

1. Verification Fundamentals (4/5)

- Verification Flow and Methodology
- Test plan creation
- Testbench development
- Test execution
- Result analysis
- Verilog Testbench vs. C++ Testbench Comparison

1. Verification Fundamentals (5/5)

- Similarities
- Differences
- When to use each
- Performance considerations

2. Testbench Architecture (1/6)

- Verilog Testbench Module Structure
- Module declaration
- Signal declarations
- DUT instantiation
- Test sequence
- Module-based organization

2. Testbench Architecture (2/6)

- C++ Testbench Structure
- Main function
- Verilator initialization
- DUT instantiation
- Test sequence
- Class-based organization (optional)

2. Testbench Architecture (3/6)

- DUT Instantiation (Both Paradigms)
- Verilog: Module instantiation
- C++: Object creation
- Signal connectivity
- Port mapping
- Signal Connectivity

2. Testbench Architecture (4/6)

- Input signals
- Output signals
- Bidirectional signals
- Signal naming
- Testbench Hierarchy
- Top-level testbench

2. Testbench Architecture (5/6)

- DUT hierarchy
- Testbench components
- Organization patterns
- Top-Level Testbench Organization
- Single testbench file
- Multiple file organization

2. Testbench Architecture (6/6)

- Include files
- Library organization
- ``module3/examples/verilog_testbenches/`` - Verilog testbench structure
- ``module3/examples/cpp_testbenches/`` - C++ testbench structure

3. Basic Testbench Components (1/6)

- Clock Generation (Verilog and C++)
- Verilog: Always blocks for continuous clock
- C++: Simulation loop for clock toggling
- Clock period control
- Multiple clocks
- Reset Generation (Verilog and C++)

3. Basic Testbench Components (2/6)

- Verilog: Initial blocks for reset sequences
- C++: Reset in C++ code
- Synchronous reset
- Asynchronous reset
- Reset timing
- Stimulus Generation (Both Paradigms)

3. Basic Testbench Components (3/6)

- Test vector generation
- Pattern application
- Sequential stimulus
- Timing control
- Response Monitoring (Both Paradigms)
- Output monitoring

3. Basic Testbench Components (4/6)

- Real-time monitoring
- Post-processing
- Monitoring strategies
- Result Checking (Both Paradigms)
- Expected value calculation
- Output comparison

3. Basic Testbench Components (5/6)

- Error detection
- Pass/fail reporting
- Simulation Control
- Starting simulation
- Stopping simulation
- Simulation termination

3. Basic Testbench Components (6/6)

- Control flow
- ``counter_test.v`` / ``counter_test.cpp`` - Clock and reset generation

4. Signal Access and Monitoring (1/5)

- Verilog: Signal Reading and Writing
- Direct signal access
- Signal assignment
- Signal reading
- Signal types
- C++: Signal Access via Verilator API

4. Signal Access and Monitoring (2/5)

- DUT object access
- Signal reading: ``dut->signal``
- Signal writing: ``dut->signal = value``
- Signal types
- Verilog: `$display`, `$monitor`, `$strobe`
- ``$display``: Immediate output

4. Signal Access and Monitoring (3/5)

- ``$monitor``: Continuous monitoring
- ``$strobe``: End-of-time-step output
- Formatting options
- C++: `std::cout`, Logging Libraries
- ``std::cout`` for output
- Formatting with `iomanip`

4. Signal Access and Monitoring (4/5)

- Logging libraries
- File output
- Formatting Output (Both Paradigms)
- Verilog: Format specifiers
- C++: Stream manipulators
- Table formatting

4. Signal Access and Monitoring (5/5)

- Readable output
- Timing of Display Statements
- When output occurs
- Event scheduling
- Timing considerations
- Debugging output

5. Simulation Control (1/5)

- Verilog: Time Management (# delays)
- Delay control: `#10`
- Time advancement
- Event scheduling
- Timing accuracy
- C++: Time Management (Simulation Loop)

5. Simulation Control (2/5)

- Simulation loop
- Time tracking
- Clock cycle counting
- Time advancement
- Event Scheduling Concepts
- Event-driven simulation

5. Simulation Control (3/5)

- Event queue
- Scheduling order
- Timing relationships
- Simulation Termination
- Verilog: ``$finish``
- C++: ``return`` statement

5. Simulation Control (4/5)

- Cleanup procedures
- Resource management
- Debugging Basics (Both Tools)
- Compilation debugging
- Runtime debugging
- Signal inspection

5. Simulation Control (5/5)

- Waveform analysis
- ``counter_test.v`` / ``counter_test.cpp`` - Time management

6. Simple Verification Patterns (1/5)

- Directed Testing (Both Paradigms)
- Specific test cases
- Known input/output pairs
- Exhaustive testing
- Corner case testing
- Test Vector Application

6. Simple Verification Patterns (2/5)

- Test vector format
- Vector application
- Sequential application
- Batch testing
- Expected vs. Actual Comparison
- Expected value calculation

6. Simple Verification Patterns (3/5)

- Actual value reading
- Comparison logic
- Error detection
- Pass/Fail Reporting
- Test result tracking
- Pass/fail determination

6. Simple Verification Patterns (4/5)

- Result reporting
- Summary generation
- Basic Error Detection
- Error detection methods
- Error reporting
- Error handling

6. Simple Verification Patterns (5/5)

- Debugging aids

7. Paradigm Comparison

- See docs/MODULE3.md

Practice & assessment

What you should know (1/8)

- Understand testbench purpose and structure
- Create basic Verilog testbench modules
- Create basic C++ testbenches
- Generate clocks and resets in both paradigms
- Apply stimulus to DUT (both paradigms)
- Monitor DUT outputs (both paradigms)

What you should know (2/8)

- Perform basic result checking
- Control simulation execution
- Choose appropriate testbench paradigm
- Use ``and_gate_test.v`` as reference
- Test all input combinations
- Verify correct operation

What you should know (3/8)

- Use ``and_gate_test.cpp`` as reference
- Compare with Verilog version
- Understand differences
- Create Verilog testbench
- Create C++ testbench
- Compare approaches

What you should know (4/8)

- Implement clock generation
- Implement reset sequences
- Compare timing approaches
- Review comparison guide
- Understand when to use each
- Identify key differences

What you should know (5/8)

- Can explain testbench purpose and structure
- Can create basic Verilog testbench modules
- Can create basic C++ testbenches
- Can generate clocks and resets (both paradigms)
- Can apply stimulus to DUT (both paradigms)
- Can monitor DUT outputs (both paradigms)

What you should know (6/8)

- Can perform basic result checking
- Can control simulation execution
- Can choose appropriate testbench paradigm
- Prerequisites: [Module 1: iverilog Deep Dive](MODULE1.md) and [Module 2: Verilator Deep Dive](MODUL...)
- Next Steps: [Module 4: Basic Testbench Construction](MODULE4.md) - Master structured testbench cons...
- Advanced: [Module 6: SystemVerilog Testbench Features](MODULE6.md) - Advanced testbench features

What you should know (7/8)

- UVM Connection: [UVM Core Repository](<https://github.com/universal-verification-methodology/core>)
- Module 4: Basic Testbench Construction - Master structured testbench construction
- Module 5: Procedural Testbench Writing - Learn procedural testbench patterns
- Icarus Verilog Documentation: <http://iverilog.wikia.com/>
- Verilator Documentation: <https://verilator.org/>
- GTKWave Documentation: <http://gtkwave.sourceforge.net/>

What you should know (8/8)

- IEEE 1364-2005 Standard: Verilog Hardware Description Language

Assessment checklist

- Can explain testbench purpose and structure
- Can create basic Verilog testbench modules
- Can create basic C++ testbenches
- Can generate clocks and resets (both paradigms)
- Can apply stimulus to DUT (both paradigms)
- Can monitor DUT outputs (both paradigms)

Summary & next steps

- Understand testbench architecture and basic verification concepts for both Verilog and C++ testbench...
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: Next module in course